به نام خدا

گزارش پروژهٔ درس رایانش ابری

hermes-mini

ربات تلگرامی بدون سرور برای مدیریت کارهای روزانه
با فهم متن آزاد فارسی، روی پلن رایگان Cloudflare

نام و نام خانوادگی	امیرعلی ابطحی
شمارهٔ دانشجویی	4021277323
درس	رایانش ابری
تاریخ	آبان 1405

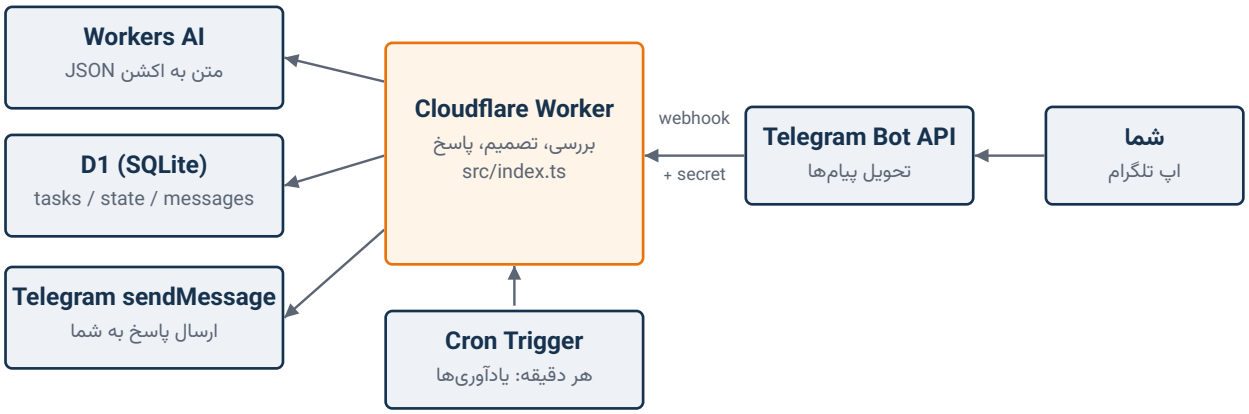
۱. خلاصهٔ پروژه

`hermes-mini` رباتی در تلگرام است که کارهای روزانه را از روی پیام‌های آزاد فارسی ثبت، تیک، حذف و جست‌وجو می‌کند؛ بدون هیچ دستوری. کاربر مثل گپ با یک آدم می‌نویسد «شیر بخر» یا «شیر رو خریدم» و ربات می‌فهمد. یک مدل زبانی فقط تشخیص می‌دهد کاربر چه می‌خواهد، و برنامهٔ من (Worker) پیشنهاد مدل را بررسی می‌کند و اگر امن بود اجرا می‌کند.

وضعیت در D1 ذخیره می‌شود، پس با راه‌اندازی دوباره از بین نمی‌رود و از هر دستگاهی در دسترس است. کل سیستم روی پلن رایگان Cloudflare (Workers، D1 و Workers AI) اجرا می‌شود و هیچ سرور شخصی‌ای ندارد. قابلیت امتیازی یادآوری روزانه هم با Cron Trigger پیاده شده است: هر صبح فهرست امروز و سر ساعت هر کار یک یادآوری.

> مخزن کد: github.com/Amirali207/hermes-mini
> آدرس سرویس: hermes-mini.zjyh04fkr-l5m0lx456tvryhkk3x.workers.dev

۲. معماری نهایی



مسیر یک پیام

1. تلگرام پیام را با یک درخواست `POST` به `/webhook` می‌فرستد و یک هدر مخفی کنارش می‌گذارد. اگر هدر نادرست یا خالی باشد، Worker جواب **403** می‌دهد.
2. شناسهٔ پیام (`update_id`) ثبت می‌شود تا ارسال دوبارهٔ تلگرام نادیده گرفته شود.
3. پیام کسی جز کاربر مجاز بی‌صدا نادیده گرفته می‌شود.
4. اگر ربات منتظر جواب سؤالی است (ساعت کار، تأیید حذف، تأیید کار تکراری)، اول همان بررسی می‌شود.
5. در غیر این صورت، متن به همراه فهرست شمارهٔ کارهای امروز و چند پیام آخر گفتگو به مدل می‌رود و مدل یک خط JSON برمی‌گرداند.
6. Worker خروجی را اعتبارسنجی می‌کند، کار را در D1 انجام می‌دهد و پاسخ را می‌فرستد. Worker همیشه به تلگرام **200** می‌دهد تا خطاها فقط لاگ شوند و درخواست دوباره‌ارسال نشود.

تصمیم‌های طراحی (فاز ۱)

پرسش	تصمیم و دلیل
Webhook یا Polling؟	**Webhook.** تلگرام فقط وقتی پیامی هست Worker را صدا می‌زند، اما polling به پردازه‌ای نیاز دارد که تمام روز روشن باشد و مدام بپرسد «پیام تازه‌ای هست؟». با webhook می‌توانم فرستنده را هم با هدر مخفی احراز کنم.
چرا FaaS؟	بار کاری چند پیام در روز است و هر کدام چند میلی‌ثانیه کار دارد؛ یعنی سیستم تقریباً همیشه بیکار است. یک ماشین مجازی یا کانتینر در همین مدت هم هزینه و هم مدیریت می‌خواست. Workers بدون مدیریت سرور، HTTPS، نگهداری secret، لاگ و cron را هم‌زمان می‌دهد.

چرا D1 و نه KV؟	ربات سؤال‌های رابطه‌ای می‌پرسد: کارهای یک روز، مرتب‌سازی بر اساس ساعت، جست‌وجوی متنی در همهٔ روزها، حذف چند id و شمارش. در D1 هرکدام یک دستور SQL است. در KV برای هرکدام باید ایندکس دستی ساخت و بارها خواند. داده هم کوچک و جدولی است.
state کجا است؟	فقط در Worker .D1 بین دو درخواست هیچ چیزی در حافظه نگه نمی‌دارد، پس هرچه باید بماند ذخیره می‌شود: جدول `tasks` ، جدول `state` (سؤال‌های در انتظار، شناسهٔ پیام‌های دیده‌شده، نشانهٔ پیام صبحگاهی) و جدول `messages` (حافظهٔ کوتاه گفتگو).
«امروز» یعنی چه؟	تاریخ تقویمی به وقت **تهران** (`Asia/Tehran` ، یعنی UTC+3:30 و بدون ساعت تابستانی از ۱۴۰۱). Worker در UTC اجرا می‌شود و منطقهٔ زمانی ندارد، پس تاریخ و ساعت در هر درخواست با `Intl.DateTimeFormat` و `timeZone: 'Asia/Tehran'` حساب می‌شود. زمان‌بندی cron هم UTC است؛ برای همین هر دقیقه اجرا می‌شود و خود کد ساعت تهران را چک می‌کند.

۳. لایهٔ هوش مصنوعی؛ چطور امن می‌ماند

مدل هیچ‌وقت به دیتابیس دست نمی‌زند. فقط یکی از اکشن‌های ثابت (`add, done, undone, settime, del, delall, list, search, chat`) را پیشنهاد می‌دهد و تصمیم نهایی با کد من است. با خروجی مدل مثل ورودی یک غریبه رفتار می‌کنم:

- **پارس و اعتبارسنجی سخت‌گیرانه.** JSON استخراج و پارس می‌شود، اکشن باید در فهرست مجاز باشد و نوع و طول هر فیلد چک می‌شود (متن کار تا 200 حرف، حداکثر 10 کار در هر پیام، ساعت حتماً `HH:MM` معتبر). شمارهٔ کارها با فهرست واقعی مقایسه می‌شود. هر ایرادی یعنی «متوجه نشدم» و **هیچ چیزی نوشته نمی‌شود** (نیازمندی R3). نسخهٔ اولی که از کد مرجع گرفتم در صورت شکست، متن خام را کار جدید ثبت می‌کرد؛ مسیر شکست حالا چیزی نمی‌نویسد.
- **عملیات مخرب باید مستند به پیام باشد.** حذف فقط وقتی فوراً انجام می‌شود که پیام کار را مشخص کند (شماره یا کلمه‌ای از متنش). در غیر این صورت، و همیشه برای «حذف همه» یا چند کار، ربات فهرستِ آنچه حذف می‌شود را نشان می‌دهد و منتظر «بله» می‌ماند. تأیید بعد از 5 دقیقه منقضی می‌شود و هر پیام نامربوط آن را لغو می‌کند.
- **کار تکراری.** همان متن با همان ساعتِ یک کار باز امروز، یک سؤال بله/نه ایجاد می‌کند؛ ساعت متفاوت یعنی کار متفاوت. ارسال دوبارهٔ تلگرام جداگانه با ثبت `update_id` مدیریت می‌شود (حداکثر یک‌بار پردازش: گم شدن یک پیام بهتر از دو بار انجام شدن آن است).
- **جواب ساعت سخت‌گیرانه چک می‌شود.** پاسخ به «ساعتش چنده؟» فقط وقتی پذیرفته می‌شود که چیزی جز کلمه‌های ساعت نداشته باشد، تا کار جدید به‌اشتباه جواب ساعت حساب نشود.
- **دسترسی و رمزها.** فقط شناسهٔ مجاز پاسخ می‌گیرد. توکن ربات و secret مربوط به webhook فقط در secret store ی Cloudflare هستند و در مخزن نیستند.

۴. داده‌ها و استقرار

تغییرات ساختار دیتابیس، جدا از کد و به‌صورت migration های شماره‌دار در پوشهٔ `migrations/` نگه‌داری می‌شود: فایل `0001` جدول `tasks` را می‌سازد، `0002` ستون‌های ساعت و یادآوری و جدول `state` را اضافه می‌کند و `0003` جدول `messages` را. استقرار با `wrangler deploy` انجام می‌شود، observability روشن است و هنگام کار با `wrangler tail` لاگ زنده را می‌بینیم. مسیر ریشه (`GET /`) جواب «hermes-mini is up» را می‌دهد و نقش health check را دارد؛ یک `POST` بدون هدر درست به webhook جواب 403 می‌گیرد. نام مدل یک متغیر است (`AI_MODEL`)، پس عوض کردنش به تغییر کد نیاز ندارد.

۵. تحلیل هزینه در دو مقیاس

روش و فرض‌ها. نمی‌توانستم 10,000 کاربر واقعی را اندازه بگیرم، پس جدول زیر یک **تخمین** بر پایهٔ نرخ‌های منتشرشدهٔ Cloudflare و فرض‌های اعلام‌شده است. نرخ‌ها را «همان‌طور که هنگام بررسی دیدم» بخوانید و پیش از اتکا دوباره چک کنید. فرض‌ها: 30 پیام در روز برای 1 کاربر و 10 پیام در روز برای هر کدام از 10,000 کاربر؛ هر پیام حدود 1,800 توکن ورودی (پرامپت فارسی به‌همراه فهرست کارها و تاریخچه) و حدود 60 توکن خروجی دارد؛ هر پیام حدود 10 ردیف خواندن و 3 ردیف نوشتن در D1. سقف‌های رایگان: Workers برابر 100,000 درخواست در روز؛ D1 برابر 5 میلیون ردیف خواندن و 100,000 ردیف نوشتن در روز؛ Workers AI برابر 10,000 neuron در روز. هزینهٔ Workers AI در پلن پولی حدود 0.011 دلار برای هر 1,000 neuron است.

مورد	1 کاربر (پلن رایگان)	10,000 کاربر
پیام در روز	حدود 30	حدود 100,000
درخواست Workers در روز	حدود 1,470 (30 پیام + 1,440 اجرای cron)؛ یعنی 1.5% سقف رایگان	حدود 101,440؛ درست روی سقف رایگان، پس باید به Workers Paid رفت (پایهٔ 5 دلار در ماه)
ردیف‌های D1 در روز	حدود 300 خواندن برای پیام‌ها و کوئری cron روی چند ردیف؛ خیلی کمتر از سقف	حدود 1 میلیون خواندن و 300,000 نوشتن؛ **نوشتن از سقف رایگان (100,000) بیشتر می‌شود**، خواندن مشکلی ندارد
neuron هر پیام	مدل Qwen3-30B-A3B: حدود 10 | مدل Llama-3.3-70B: حدود 60 (در هر دو مقیاس یکسان)	
neuron در روز	Qwen: حدود 300 (3% سقف رایگان) | Llama 70B: حدود 1,800 (18%)	Qwen: حدود 1 میلیون | Llama 70B: حدود 6 میلیون
پیام‌هایی که سهمیهٔ رایگان AI در روز پوشش می‌دهد	Qwen: حدود 985 | Llama 70B: حدود 165	بین 0.2% تا 1% نیاز واقعی
هزینهٔ تخمینی AI	0 (سهمیهٔ رایگان)	Qwen: حدود 11 دلار در روز (حدود 335 دلار در ماه) | Llama 70B: حدود 66 دلار در روز (حدود 1,990 دلار در ماه)

اول کدام سقف می‌شکند و چه چیزی را عوض می‌کردم؟

- **سهمیهٔ هوش مصنوعی اول می‌شکند و انتخاب مدل مهم‌ترین اهرم هزینه است.** مدل 70 میلیارد پارامتری برای هر پیام حدود شش برابر مدل 30 میلیارد پارامتری هزینه دارد؛ با این تخمین، با مدل 70B یک جلسهٔ تست طولانی (حدود 165 پیام) کل سهمیهٔ رایگان روزانه را مصرف می‌کند. کوتاه‌تر کردن پرامپت و کوچک‌تر کردن مدل مستقیماً هزینه را پایین می‌آورد.
- **cron ی هر دقیقه، همان‌طور که نوشته شده، مقیاس‌پذیر نیست.** کوئری یادآوری روی روز فیلتر می‌کند، اما تنها ایندکس موجود روی `day` است، پس هر بار همهٔ کارهای آن روز اسکن می‌شود. با 10,000 کاربر و حدود 5 کار برای هرکدام، یعنی ≈ 1,440 × 50,000 72,000,000 ردیف خواندن در روز، که از سقف 5 میلیونی رایگان بسیار بیشتر است. یک ایندکس ترکیبی روی `(day, reminded, time)` باعث می‌شود هر اجرا فقط ردیف‌های موعدرسیده را بخواند.
- **طراحی فعلی تک‌کاربره است.** برای چندکاربره باید به همهٔ جدول‌ها `user_id` اضافه شود، کلیدهای state به‌ازای هر کاربر باشند و منطقهٔ زمانی هر کاربر جدا ذخیره شود. تلگرام هم ارسال پیام ربات را حدود 30 پیام در ثانیه محدود می‌کند؛ یعنی پیام صبحگاهی برای 10,000 کاربر دست‌کم 5 تا 6 دقیقه طول می‌کشد و باید از صف یا ارسال پله‌پله استفاده کرد.

۶. کجا گیر کردم و چطور حل کردم

بیشتر مشکل‌ها از خود ایده نبودند؛ از ابزار، از state و از اعتماد زیادِ من به مدل بودند. روشی که هر بار جواب داد: خطا را قابل‌دیدن کنم (لاگ زنده و کد خطا)، آنچه Cloudflare واقعاً دارد را با آنچه کد فرض کرده مقایسه کنم، یک چیز را درست کنم و دوباره تست کنم.

مشکل	چه می‌دیدم	چطور حلش کردم
نصب بسته‌ها نمی‌شد	`npm install` با خطای ECONNRESET قطع می‌شد.	مشکل شبکه بین سیستم من و رجیستری npm بود (فیلتر یا مسیر ناپایدار). با درست کردن تنظیم اتصال و تلاش دوباره حل شد؛ npm بسته‌های دانلودشده را نگه می‌دارد.
دیتابیس اشتباه در تنظیمات	migration خطای «دیتابیس پیدا نشد» (کد 7404) می‌داد. Wrangler کنار ورودی قدیمیِ مرجع یک binding دوم با نام جدید اضافه کرده بود.	فقط یک ورودی با `database_id` خودم و نام binding ی که کد استفاده می‌کند (DB) نگه داشتم. درس: Wrangler دیتابیس را با نامِ داخل تنظیمات پیدا می‌کند، نه از روی حساب.
همه‌چیز «کار» ثبت می‌شد	هر پیامی، حتی سلام، با «اضافه شد» جواب می‌گرفت.	نسخهٔ اول کد مرجع هر وقت نتیجهٔ AI قابل‌استفاده نبود، متن خام را کار ثبت می‌کرد و هر خرابی AI را هم پنهان می‌کرد. خطاها را با `wrangler tail` قابل‌دیدن کردم، پارسر را طوری نوشتم که برای هر چیز نامعتبر `null` بدهد و به‌جای ثبت، «متوجه نشدم» بگوید.

شکاف در schema	کد جدید به `time` و `reminded` و `state` نیاز داشت، اما D1 راه‌دور من فقط migration اول را داشت.	جدول‌های واقعی را با `PRAGMA table_info` بررسی کردم، در مخزن migration ی برایشان پیدا نکردم و `0002` را اضافه کردم، به‌جای ویرایش migration ی که قبلاً اعمال شده بود.
خطای دسترسی در ویندوز	Wrangler نمی‌توانست پوشهٔ cache را بنویسد؛ حتی ساختن یک پوشه در مخزن رد می‌شد.	ACL پوشه را خواندم (کاربر معمولی من فقط دسترسی خواندن داشت، احتمالاً به‌خاطر کلون قبلی با دسترسی ادمین) و در یک پوشهٔ تمیز، از یک shell معمولی (نه ادمین) دوباره کلون کردم.
درخواست AI رد می‌شد	خطای 5006: ورودی `messages/1` باید role و content داشته باشد.	در قابلیت تاریخچه به‌جای `...history` نوشته بودم `history` و یک آرایه را داخل آرایهٔ دیگر گذاشته بودم. اصلاح کردم و حالا ردیف‌های ذخیره‌شده را هم فیلتر می‌کنم تا یک ردیف خراب نتواند درخواست را بشکند.
تلگرام از ترمینال در دسترس نبود	`curl` به API تلگرام timeout می‌خورد، در حالی که خود ربات کار می‌کرد.	تلگرام در ایران فیلتر است و ترمینال من از VPN رد نمی‌شد. Worker تأثیری نمی‌پذیرد چون از شبکهٔ Cloudflare به تلگرام وصل می‌شود؛ پس به‌جای آن از طریق خود ربات و لاگ‌های Worker بررسی کردم.

تست پیام‌های چالشی چه چیزی را نشان داد

پیام‌های سخت به سبک بازبینی را به ربات فرستادم و اولین دور چهار ایراد واقعی پیدا کرد که تست مسیر معمولی هیچ‌وقت نشان نمی‌داد:

- **«همه رو حذف کن» همه‌چیز را پاک کرد** وقتی فهرست دو کار یا کمتر داشت. اولین محافظ من فقط درخواست‌هایی با بیش از دو شماره را رد می‌کرد. راه‌حل: هر حذفِ چندکاره تأیید می‌گیرد.
- **مدل حدس می‌زد.** با «حذفش کن» یا «اون یکی» کاری را انتخاب می‌کرد و شمارهٔ معتبر پذیرفته می‌شد، حتی وقتی کاربر کار را مشخص نکرده بود. راه‌حل: عملیات مخرب باید به خود پیام تکیه کند، وگرنه ربات می‌پرسد.
- **کار جدید به‌جای جواب ساعت گرفته می‌شد.** وقتی ربات منتظر ساعت بود، پیامی مثل «مطالعه بیو ساعت 8» ساعت 08:00 را روی کار قبلی گذاشت. راه‌حل: فقط پاسخی که صرفاً ساعت باشد جواب ساعت حساب می‌شود.
- **کار تکراری.** یک کار دو بار ثبت می‌شد. راه‌حل: تأیید برای همان متن و همان ساعت.

آنچه یاد گرفتم

- خروجی مدل زبانی را باید مثل ورودی کاربر اعتبارسنجی کرد؛ «شمارهٔ معتبر» با «شمارهٔ درست» فرق دارد.
- در serverless هر چیزی که باید بماند باید ذخیره شود، و ابزارهای مشاهده (`wrangler tail`، کد خطا، `PRAGMA`) سریع‌ترین راه پیدا کردن خطا هستند.
- هزینه را باید از همان ابتدا با عدد و فرض نوشت؛ سهمیهٔ هوش مصنوعی زودتر از بقیهٔ سقف‌ها به مرز می‌رسد.

استفاده از ابزار هوش مصنوعی. همان‌طور که درس اجازه می‌دهد، از یک دستیار هوش مصنوعی به‌عنوان مربی و برای رفع اشکال و پیش‌نویس استفاده کردم. هر تغییرِ توصیف‌شده در این گزارش را اعمال و مرور کردم، روی ربات واقعی تست کردم و می‌توانم معماری و کد را در جلسهٔ دمو توضیح بدهم.